

PVD Model	Training Epochs	Final Loss ↓
SVG-to-PVD (Qwen2.5-7B)	3 epochs	0.052
SVG-to-PVD (Mistral-7B)	3 epochs	0.051
PNG-to-PVD (Qwen2.5-VL-7B)	3 epochs	0.243

Table 9: Comparison of training loss between the PNG-to-PVD and SVG-to-PVD models.

PVD Model	SSIM ↑	DINOv2 Score ↑	CLIP Score ↑
SVG-to-PVD (Mistral-7B)	0.895	0.880	0.893
SVG-to-PVD (Qwen2.5-7B)	0.889	0.876	0.880
PNG-to-PVD (Qwen2.5-VL-7B)	0.262	0.320	0.385

Table 10: Comparison of perception metrics between the PNG-to-PVD and SVG-to-PVD models.

Ablation with PNG-to-PVD model. To further validate the design choice of using SVG for initial visual encoding, we compare it with directly training a PNG-to-PVD model using a large multimodal model. Specifically, we train Qwen2.5-VL-7B^{||} on the same PVD-160k dataset, where the inputs are png images and the target outputs are PVD strings. Table 9 and Table 10 show the comparison between the PNG-to-PVD and SVG-to-PVD models in terms of final training loss and perception metrics. Using identical hyperparameters, we find that directly translating PNG to PVD is significantly less effective, as evidenced by a much higher loss and substantially lower perception performance. We observe that the PNG-to-PVD model rarely produces valid PVD JSON outputs and yields significantly worse reconstruction performance (entirely not usable for downstream reasoning).

F Additional Experiments Using Open-Source LMMs as Reasoners

Reasoner	PVD Model	AC	LC	SW-S 2Obj	SW-S mObj	SW Sup	NLVR	Geo	Maze 2×2	Maze 3×3	All
Qwen2.5-VL-72B	–	0.78	0.75	0.98	0.75	0.92	0.72	0.67	0.64	0.15	0.707
Qwen2.5-VL-72B	Mistral-7B	0.59	0.96	0.96	0.78	0.89	0.69	0.73	0.66	0.28	0.727

Table 11: End-task performance with open-source LMM reasoners.

We added new experimental results demonstrating that VDLM can also work effectively with recent open-source LMMs. Specifically, we use Qwen-2.5-VL-72B^{**} as the reasoner to compare performance with and without the inclusion of PVD representations. Across 9 tasks, we observe a 2% overall improvement.

G Additional Qualitative Examples on PVD Parsing Novel Concepts

We highlight that our PVD ontology constitutes a minimal but powerful set of primitives for expressing shapes in vector graphics. We demonstrate that the current PVD model shows promise in approximating novel shapes through composition. In Figure 16, we show several examples of the PVD model parsing novel concepts such as “star,” “cross,” and “circle segment.”

H Further Exploration in Prompt Engineering

Observing that the PVD representation can sometimes be inaccurate and may degrade performance (i.e., on ShapeWorld tasks), we explore introducing a verification step in the prompt. Specifically, we instruct the model to first “double-check whether the objects in the PVD perception match the objects in the image.” We find that this step can enhance performance on tasks where the initial image-only baseline is already reasonably strong. Table 12 shows the performance comparison on ShapeWorld tasks with the same LMM reasoner and PVD representation.

^{||}<https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct>

^{**}<https://huggingface.co/Qwen/Qwen2.5-VL-72B-Instruct>

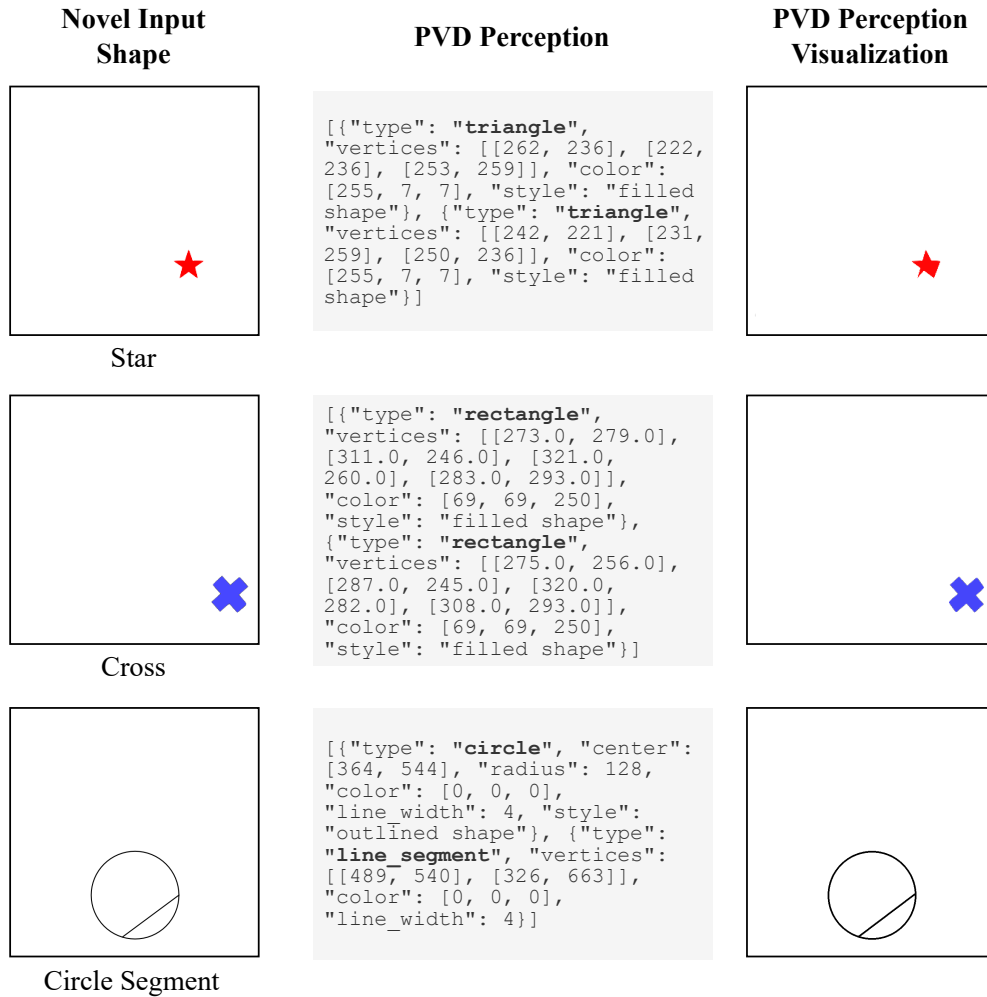


Figure 16: Qualitative examples on PVD model approximating novel concepts.

Prompt Version	Reasoner	PVD Model	SW-S 2Obj	SW-S mObj	SW Sup
-	GPT-4o	-	0.97	0.81	0.92
Original	GPT-4o	Mistral-7B	0.91	0.82	0.82
New	GPT-4o	Mistral-7B	0.98	0.81	0.88

Table 12: ShapeWorld task performance with new prompt.

I Full Response of the Example in Figure 2

See Figure 17 for the full input prompt and the generated response from GPT-4 on the 2×2 maze-solving task shown in Figure 2.

J Task Prompts

Figure 18 shows the prompts for models with only image representations as visual inputs.

Figures 19-27 show the prompts for VDLM, where {perception} will be filled with the aggregated Primal Visual Description perception result, and the orange text are instance-specific inputs such as the question. For VDLM-mm, the original image input will be preserved and feed to the LMM reasoner along with the

filled prompt. Since the reasoning in VDLM-txt is based solely on the PVD representation which is purely textual, task instructions that assume visual inputs can become ambiguous. For example, in the task Angle Classification, it is unclear which angle the question is referring to if we are only given the coordinates of two undirected edges. Therefore, we design task-specific prompts that remove such ambiguity. Another noteworthy point is that, in contrast to visual inputs that naturally accommodate a degree of imprecision, symbolic representations lack such inherent leniency. For instance, even if two line segments differ by only one pixel in length, they might be considered identical in visual representations, but symbolic representations would likely identify them as different. To reintroduce a level of tolerance in tasks that involve arithmetic reasoning, such as length comparison, we incorporate task-specific instructions to account for a reasonable margin of error, like 5%.

K Newly Constructed Downstream Task Datasets

Angle Classification. We use the Geoclidian data generator^{††} to generate images containing a single acute or obtuse angle with randomized orientations and ray lengths. The domain-specific language for generating the two concepts are shown as follows:

- Acute Angle:

```
"l1* = line(p1(), p2())",
"c1* = circle(p1(), p2())",
"c2* = circle(p2(), p1())",
"l2* = line(p3(c1, c2), p4(c1, c2))",
"l4 = line(p5(l1, l2), p7(l1))",
"l5 = line(p6(l2), p7(l1))"
```

- Obtuse Angle:

```
"l1* = line(p1(), p2())",
"c1* = circle(p1(), p2())",
"c2* = circle(p2(), p1())",
"l2* = line(p3(c1, c2), p4(c1, c2))",
"l3* = line(p5(l1, l2), p6(l2))",
"l4* = line(p5(l1, l2), p7(l1))",
"l5* = line(p6(l2), p7(l1))",
"l6* = line(p8(l3, l4), p9(l5))",
"l100* = line(p5(c1, c2), p10(l6))",
"c101* = circle(p5(c1, c2), p10(l6))",
"c102* = circle(p10(l6), p5(c1, c2))",
"l101* = line(p100(c101, c102),
p101(c101, c102))",
"l7 = line(p11(l100, l101), p6(l2))",
"l8 = line(p11(l100, l01), p7(l1))"
```

Length Comparison. We use matplotlib^{‡‡} to plot two non-intersecting line segments on a canvas. These line segments may either be of identical length or of differing lengths. In scenarios where the lengths vary, we ensure the discrepancy is substantial (exceeding 15% relative to the length of the shorter line segment) to ensure perceptibility. The orientation of each line segment is determined independently and randomly, being either horizontal or vertical.

^{††}https://github.com/joyhsu0504/geoclidian_framework

^{‡‡}<https://matplotlib.org/stable/>